Specification

Specification · Filter catalog: cheap AND in-stock · Build with Waqar

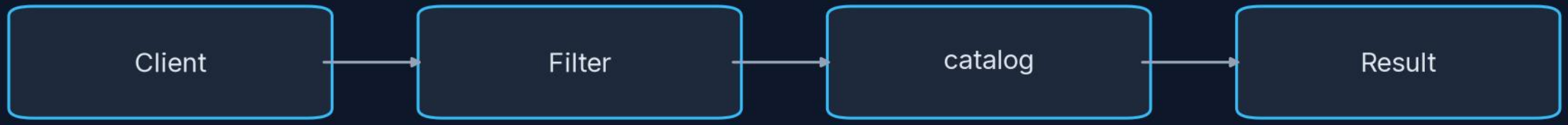
Today's Hook

- Today's force: Filter catalog: cheap AND in-stock
- We earn Specification by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Product, Spec, CheapSpec, InStockSpec

The Problem

- Scenario: Filter catalog: cheap AND in-stock
- Constraints: solve «Filter catalog: cheap AND in-stock» with Specification — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Filter catalog: cheap AND in-stock
- Success looks like: [item0]

Structure



Specification: Filter catalog: cheap AND in-stock

Real-World Scenario

- Running example: Filter catalog: cheap AND in-stock
- Key types: Product, Spec, CheapSpec, InStockSpec
- Happy-path stdout cue: [item0]
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.specification.problem;

import java.util.*;

public final class NaiveFilter {
  public static List<String> filter(List<int[]> items){ // [price, stock]
    List<String> out=new ArrayList<>();
    for (int i=0;i<items.size();i++) if(items.get(i)[0]<1000 && items.get(i)[1]>0) out.add("item"+i);
    return out;
  }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Filter catalog: cheap AND in-stock
- Without Specification, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.enterprise.specification.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

record Product(String id, int priceCents, int stock){}
interface Spec { boolean isSatisfiedBy(Product p); default Spec and(Spec o){ return p -> this.isSatisfiedBy(p) && o.isSatisfiedBy(p); } }
final class CheapSpec implements Spec { private final int max; CheapSpec(int max){this.max=max;} public boolean isSatisfiedBy(Product p){ return p.priceCents()<max; } }
final class InStockSpec implements Spec { public boolean isSatisfiedBy(Product p){ return p.stock()>0; } }
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.specification.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Specification: cheap AND in-stock ===");
    System.out.println("--- naive ---"); System.out.println(NaiveFilter.filter(List.of(new int[]{500,1}, new int[]{2000,5}, new int[]{100,0})));
System.out.println("--- specification ---");
var catalog=List.of(new Product("a",500,1), new Product("b",2000,5), new Product("c",100,0));
Spec spec=new CheapSpec(1000).and(new InStockSpec());
System.out.println(catalog.stream().filter(spec::isSatisfiedBy).map(Product::id).toList());
  }
}
```

Walkthrough Output

```
=== Specification: cheap AND in-stock ===
--- naive ---
[item0]
--- specification ---
[a]
```

Step-by-Step Trace

- Match stdout to stages — look for: [item0]
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 29-specification ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
package com.buildwithwaqar.patterns.enterprise.specification.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

record Product(String id, int priceCents, int stock){}
interface Spec { boolean isSatisfiedBy(Product p); default Spec and(Spec o){ return p -> this.isSatisfiedBy(p) && o.isSatisfiedBy(p); } }
final class CheapSpec implements Spec { private final int max; CheapSpec(int max){this.max=max;} public boolean isSatisfiedBy(Product p){ return p.priceCents()<max; } }
final class InStockSpec implements Spec { public boolean isSatisfiedBy(Product p){ return p.stock()>0; } }
```

Trade-offs

- Use Specification when «Filter catalog: cheap AND in-stock» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Specification in product language.

Related Patterns

- Name the pattern usually confused with Specification and separate the forces.
- Sketch the roles for «Filter catalog: cheap AND in-stock» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Filter catalog: cheap AND in-stock
- Naive pain → Specification boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.